

Singleton Pattern

Experimental Results Report

Mehedi Hasan (22)

CSE-3212 Design Patterns Assignment
MongoDB Connection Manager — Six Implementations

1. Instance Validation

Each implementation was instantiated three times. The `id()` of each returned object was recorded. Identical IDs across all three calls confirm that only one instance was ever created.

Class	ID – Call 1	ID – Call 2	ID – Call 3	Result
EagerMongoConnection	2451652573392	2451652573392	2451652573392	✓ Single instance
LazyMongoConnection	2451652015952	2451652015952	2451652015952	✓ Single instance
SynchronizedMongoConnection	2451724907472	2451724907472	2451724907472	✓ Single instance
DoubleCheckedMongoConnection	2451725015120	2451725015120	2451725015120	✓ Single instance
BillPughMongoConnection	2451725074512	2451725074512	2451725074512	✓ Single instance
EnumMongoConnection	2451724578896	2451724578896	2451724578896	✓ Single instance

Observation: All six implementations returned the same memory address across all three calls, confirming correct Singleton behavior. The IDs differ between implementations only because each class maintains its own independent instance.

2. Thread Safety Analysis

Ten concurrent threads were spawned simultaneously for each implementation. Each thread called the constructor and recorded the `id()` of the returned object. The number of unique IDs reveals whether multiple instances were accidentally created during the race window. A 50 ms artificial sleep was embedded inside the lazy implementations to widen the race window and make failures clearly visible.

Class	Unique IDs	Outcome
EagerMongoConnection	1	✓ THREAD-SAFE
LazyMongoConnection	1	✓ THREAD-SAFE

SynchronizedMongoConnection	1	✓ THREAD-SAFE
DoubleCheckedMongoConnection	1	✓ THREAD-SAFE
BillPughMongoConnection	1	✓ THREAD-SAFE
EnumMongoConnection	1	✓ THREAD-SAFE

Observations

Eager Initialization — The instance is created at module import time (before any thread runs), so there is no race window. Always produces one unique ID.

Lazy Initialization — Despite showing one unique ID in this run, this implementation has no lock. The 50 ms sleep is specifically designed to expose the race condition; in repeated runs or a heavier load, multiple instances would appear. It is not considered thread-safe.

Synchronized Method — Acquires a lock on every call. Guaranteed to produce one unique ID under any concurrency scenario.

Double-Checked Locking — The outer if-check avoids the lock after initialization; the inner check inside the lock prevents duplicate creation during first-access races.

Bill Pugh Adaptation — Uses a private nested helper class. The instance is stored in that class's attribute, and a lock guards the first assignment — thread-safe by design.

Enum-based Singleton — Enum members are class-level singletons created at definition time. No locking needed; no race condition possible.

3. Performance Measurements

3a. Initialization Time (First Call)

Each singleton was reset to a null state, then the first call was timed using `time.perf_counter()`. The 50 ms sleep is still present in implementations 2–5 to keep results comparable with the thread-safety test.

Class	Init Time (ms)	Note
EagerMongoConnection	0.005 ms	Module load time (import)
LazyMongoConnection	560.294 ms	Lazy + 50 ms sleep
SynchronizedMongoConnection	397.543 ms	Lock overhead + 50 ms sleep
DoubleCheckedMongoConnection	178.265 ms	Double check + 50 ms sleep
BillPughMongoConnection	213.841 ms	Helper class + 50 ms sleep
EnumMongoConnection	0.003 ms	Module load time (import)

Eager and Enum register near-zero initialization times because their MongoClient was constructed at import time — the timed call merely returns an already-existing reference. The remaining four implementations all include the 50 ms artificial delay, which dominates the measured time; removing it in production code would bring all four down to the actual MongoDB connection latency (approximately 150–200 ms on this network).

3b. Repeated-Access Time (50,000 calls)

With all instances already initialized, each getter was called 50,000 times using `timeit`. The average per-call cost in microseconds is reported below.

Class	Avg per Call (µs)	Interpretation
EagerMongoConnection	0.1082 µs	Simple attribute lookup
LazyMongoConnection	0.1242 µs	Simple attribute lookup
SynchronizedMongoConnection	0.3349 µs	Lock acquired every call
DoubleCheckedMongoConnection	0.1110 µs	Outer if short-circuits lock
BillPughMongoConnection	0.1240 µs	Similar to Double-Checked
EnumMongoConnection	0.1289 µs	Enum member access

The Synchronized Method is the clear outlier at 0.3349 µs per call — over three times slower than the others — because it acquires and releases a threading lock on every single access, even after the singleton is long since initialized. All other implementations converge at approximately 0.11–0.13 µs, reflecting a simple Python attribute lookup with no lock overhead.

4. Comparative Summary

The table below consolidates all experimental findings into a single side-by-side comparison.

Implementation	Thread-Safe	Init Time	Access Time	Recommendation
EagerMongoConnection	✓	~0 ms	0.1082 µs	Best for static environments
LazyMongoConnection	X (unsafe)	560 ms	0.1242 µs	Avoid in concurrent code
SynchronizedMongoConnection	✓	398 ms	0.3349 µs	Simple but slow repeated access
DoubleCheckedMongoConnection	✓	178 ms	0.1110 µs	Best balance: safe + fast
BillPughMongoConnection	✓	214 ms	0.1240 µs	Safe, clean structure
EnumMongoConnection	✓	~0 ms	0.1289 µs	Simplest thread-safe option

Key Findings from Data

Fastest repeated access: Eager (0.1082 µs) and Double-Checked (0.1110 µs) are statistically tied for the lowest per-call cost.

Thread-safety: Five of six implementations are reliably thread-safe. Lazy Initialization is the only one that is not; its single unique ID in this run is coincidental due to Python's GIL and timing, not a design guarantee.

Initialization overhead: Eager and Enum have effectively zero timed initialization because the connection is built at import time. For applications where startup time matters, these are preferable. For applications where the database connection should be deferred until actually needed, Double-Checked Locking offers the best trade-off between safety and per-call performance.

Slowest repeated access: Synchronized Method (0.3349 µs) — the constant lock acquisition makes it unsuitable for high-throughput scenarios.

Recommended choice for concurrent applications: Double-Checked Locking — thread-safe, near-zero repeated access cost, and lazy initialization avoids unnecessary connections at startup.